

SDK / стек

Параметр	Значение
Название приложения	Bclіc App
Android Gradle Plugin	8.10.0
minSdk	25
targetSdk	36
Kotlin	да, 1.6.0
Web View	да
Custom Tabs	да
Рекламные сети	нет
Аналитика	нет
Permissions	android.permission.INTERNET
Libraries	Unity IL2CPP (UnityEngine + native libil2cpp/libunity/libmain), UniWebView (com.onevcат.uniwebview), UniTask (Cysharp; UniTask/Addressables/DOTween/Linq/TextMeshPro), TextMeshPro, DOTween, Kotlin stdlib 1.6.0, AndroidX Browser 1.2.0 (Custom Tabs), AndroidX Core 1.1.0, AndroidX Lifecycle Runtime 2.0.0, AndroidX Interpolator 1.0.0, AndroidX VersionedParcelable 1.1.0, android.support.customtabs / support-v4 shims, bitter.jnibridge, org.fmod (FMOD), com.google.common.util.concurrent (ListenableFuture)
Подозрительные домены	longinus-zymurgy.com, g6games-10a78c7f.surge.sh
SharedPreferences	Unity PlayerPrefs (обёртка над SharedPreferences): Get/Set/HasKey/DeleteKey; в загрузчике AppLoaderNewVer хранятся флаги/состояние шлюза и WebView (hasSapphireWebViewStarted, isEmberLoadingActive, resolvedGatewayUrl/targetGatewayUrl и связанные поля). Точные имена ключей prefs в IL2CPP открытым списком не выписаны.
Есть ли клоака	да
Подозрительные слова	gateway, probe, fallback, redirect, webview, bridge, User-Agent, privacy, ActivateSapphireWebView, OpenFallbackGameScene, GatewayProbeUrl, resolvedGatewayUrl, targetGatewayUrl, gatewayPayloadText, OpenPrivacy

Как работает клоака

В начале

Снаружи это выглядит как обычная мобильная Unity-игра (package com.subwaysanta.xmassurf, на устройстве подпись «Bclіc App», категория game в манифесте). Но при запуске приложение не сразу отдаёт геймплей: сначала тихо «опрашивает шлюз» (gateway probe) на своём домене longinus-zymurgy.com.

Обман в том, что модератору и «не тому» посетителю показывают безопасную картину: снаружи шлюз отвечает редиректом на карточку Google Play, внутри APK есть запасная игровая сцена и privacy-страница под чужим названием «Color Shot», а «нужному» пользователю открывают встроенный браузер (WebView / «Sapphire WebView») с URL, который пришёл с шлюза.

Путь «боевого» пользователя заканчивается на динамической ссылке из ответа шлюза (resolvedGatewayUrl / targetGatewayUrl). В статическом коде конкретный казино/оффер URL не зашит — его присылает сервер. Без «правильного» клиентского probe longinus-zymurgy.com сейчас просто отправляет на Play Store.

Кому что показывают

Белый / обычный режим: модератор Google, бот, обычный браузер без «правильного» клиентского запроса к шлюзу. Что видно: HTTP 302 с longinus-zymurgy.com на страницу приложения в Play Store; внутри APK — запасной путь OpenFallbackGameScene (загрузка настоящей игровой сцены) и OpenPrivacy. На surge лежит

«белая» Privacy Policy с названием чужой игры «Color Shot offline game» / G6GAMES — типичная обложка для стора.

Чёрный / боевой режим: пользователь, которому шлюз вернул «разрешённый» payload. Что открывается: ActivateSapphireWebView — полноэкранный UniWebView (встроенный браузер внутри приложения) или Custom Tabs по URL из ответа сервера. Это уже не геймплей аркады, а веб-контент по удалённой ссылке.

Как приложение решает, кого куда пустить

1) Удалённый шлюз (GatewayProbeUrl → <https://longinus-zymurgy.com>). Что смотрит: сервер на стороне шлюза (в APK логика фильтра не раскрыта полностью — решение на сервере). Условие: ответ probe. Результат: ResolveGatewayProbeAnswer заполняет gatewayPayloadText / resolvedGatewayUrl / targetGatewayUrl → либо WebView, либо fallback-игра.

2) Запасной путь при ошибке/отказе. Что смотрит: удалось ли получить «боевой» URL. Условие: нет валидного ответа / негативный ответ. Результат: OpenFallbackGameScene + AccelerateFallbackGameLoading — показывают обычную игру.

3) Флаги состояния загрузчика. Что смотрит: hasSapphireWebViewStarted, isEmberLoadingActive и связанные поля AppLoaderNewVer (частично через PlayerPrefs). Условие: уже открыт WebView или идёт «ember»-загрузка. Результат: не дают дважды поднять WebView / ведут последовательность AdvanceEmberLoadingSequence.

4) User-Agent / WebView-настройки. UniWebView умеет setUserAgent / getUserAgent. Отдельного явного списка ботов в строках Assembly-CSharp не видно — фильтр, скорее всего, на стороне шлюза. В коде не видно отдельного geo/VPN-чекера на клиенте.

5) Privacy как белая обложка. OpenPrivacy / Privacy.cs + URL на surge с политикой «Color Shot» — для стора и проверок, а не как финальный оффер.

Цепочка по шагам

Шаг 1. Пользователь запускает приложение → стартует UnityPlayerActivity → зачем: стандартный вход Unity-игры → система получает splash/загрузчик AppLoaderNewVer.

Шаг 2. AppLoaderNewVer вызывает BeginGatewayProbeRoutine → зачем: тихо спросить у своего сервера, какой режим показать → уходит HTTP-запрос на GatewayProbeUrl (<https://longinus-zymurgy.com>).

Шаг 3. Шлюз смотрит запрос (IP/гео/клиент — на сервере; в APK детали фильтра не выписаны) → зачем: отделить модераторов/ботов от «боевого» трафика → клиент получает ответ.

Шаг 4. ResolveGatewayProbeAnswer разбирает ответ в gatewayPayloadText / resolvedGatewayUrl / targetGatewayUrl → зачем: превратить ответ сервера в конкретное решение → либо ссылка для WebView, либо сигнал идти в игру.

Шаг 5. Если ссылка «боевая» → ActivateSapphireWebView → зачем: открыть веб-контент внутри приложения → пользователь видит UniWebView (или Custom Tabs) с URL с сервера.

Шаг 6. Если отказ/ошибка/«белый» ответ → OpenFallbackGameScene / AccelerateFallbackGameLoading → зачем: показать безопасный контент → пользователь играет в обычную аркаду.

Шаг 7. Параллельно/по кнопке доступен OpenPrivacy → зачем: формальная privacy → открывается страница на g6games-10a78c7f.surge.sh (Color Shot / G6GAMES).

Шаг 8. Снаружи без приложения longinus-zymurgy.com отвечает 302 на карточку Play Store → зачем: даже при ручной проверке домена видна «белая» витрина → проверяющий не видит оффер.

Шаг 9. Флаги hasSapphireWebViewStarted / isEmberLoadingActive фиксируют этап → зачем: не ломать сценарий повторным probe → загрузчик ведёт AdvanceEmberLoadingSequence до финального экрана.

Технические уловки

Логика в IL2CPP, а не в Java. Клоака сидит в Assembly-CSharp (AppLoaderNewVer, Privacy), поэтому поверхностный grep по jadx Java почти ничего не находит. Зачем спрятали: усложнить разбор через обычный decompiled/sources.

UniWebView + JS bridge. Встроенный WebView с каналом UniWebViewNativeChannel / JavascriptInterface. Зачем: полный контроль над веб-страницей внутри игры, cookies, custom schemes.

Custom Tabs. androidx.browser и проверка CustomTabsService в манифесте — запасной способ открыть URL во внешнем Chrome Custom Tab. Зачем: иногда оффер удобнее открыть «как браузер», а не только во встроенном WebView.

Динамический финальный URL. В бинарнике зашит только шлюз longinus-zymurgy.com, не конечный

лендинг. Зачем: менять оффер на сервере без обновления APK в сторе.

Несовпадение брендов. Package subway santa / xmas surf, label «Bclіc App», privacy про Color Shot G6GAMES. Зачем: белая легенда для стора не совпадает с реальным веб-сценарием.

Роль подозрительных доменов

longinus-zymurgy.com — «мозг» решения (GatewayProbeUrl). На шагах 2–5: принимает probe, отдаёт payload с URL. Снаружи при обычном GET редиректит на Play Store (белая витрина). Боевой ответ приложению динамический; без клиентского протокола виден только 302 на стор.

g6games-10a78c7f.surge.sh — белая privacy/landing-обложка (не оффер). На шаге 7: статическая Privacy Policy «Color Shot offline game» по пути с package name. Нужна для стора и формального соответствия, не как казино-лендинг.

Финал для пользователя

После всех проверок «нужный» пользователь оказывается на веб-странице по URL из ответа шлюза (тип сайта — динамический: в коде не зашит казино/беттинг домен, его присылает longinus-zymurgy.com). «Ненужный» пользователь / модератор видит либо карточку Google Play (при заходе на домен в браузере), либо встроенную запасную игровую сцену + privacy Color Shot. Итоговый оффер URL без живого успешного probe из приложения не фиксируется — он приезжает только в gatewayPayload / resolvedGatewayUrl.